

第 5 週：功能模擬

本週一句話：好的測試平台會自己判分，你只要看最後一行。

本週指令

雙擊 `env.bat` 開好環境（提示字元要有 `[OSS CAD Suite]`），然後：

```
cd week05_simulation
```

每個範例都是這三條（外加一條檢查），把 `02_self_check` 換成你要跑的名字：

```
iverilog -o sim.out 02_self_check.v 02_self_check_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 02_self_check.gtkw
```

`echo %ERRORLEVEL%` 印 0 才是編譯成功——失敗時 iverilog 常常一個字都不印。

本週範例名稱：

- `01_display_family`
- `02_self_check`
- `03_timescale`
- `04_random_test`

下面每個範例的段落，都直接附了它自己的四行指令。

學完你要會什麼

- [] 知道 ``timescale` 的兩個數字各是什麼

- [] 分得出 `$display` / `$strobe` / `$monitor` / `$write`
- [] ★ 會寫自我檢查測試平台 (參考模型 + 自動比對 + 報分)
- [] 會用 `$random` 做亂數壓測
- [] 知道為什麼測試平台要用 `===` 不用 `==`

一、`timescale

```
`timescale 1ns / 1ps
//      ^^^  ^^^
//      單位  精度
```

	意思
單位	#1 代表多久 → 這裡 #1 = 1ns
精度	模擬器能分辨多細 → 這裡 #0.001 才有意義

[!WARNING] 沒寫 `timescale` 會怎樣？預設變成 1 秒。你 `week00_warmup` 的波形顯示「115 sec」就是這樣來的。每個 `.v` 檔最上面都應該寫一行。

二、四種印法

指令	什麼時候印	用在哪
<code>\$display</code>	呼叫的當下立刻印	一般訊息
<code>\$strobe</code>	等這個時間點所有事做完才印	印時脈邊緣後的最終值
<code>\$monitor</code>	參數任一個變了就自動印 (設定一次即可)	全程追蹤
<code>\$write</code>	同 <code>\$display</code> 但不換行	排版

★ 最容易被騙的地方 (範例 01 的真實輸出)

```
[display] t=5000    d=1  q=x    ★ q 還是舊值！  
[strobe ] t=5000    d=1  q=1    ★ q 已更新  
[display] t=15000   d=2  q=1    ★ 又是舊值  
[strobe ] t=15000   d=2  q=2
```

同一個時間點，`$display` 和 `$strobe` 印出不同的 `q`。

為什麼？在 `@(posedge clk)` 之後馬上 `$display`，非阻塞 `<=` 的更新還沒生效。

[!IMPORTANT] 除錯時如果覺得「印出來的值差一拍」，八成就是這個原因。解法：用 `$strobe`，或在 `$display` 前加 `#1`。

三、★ 自我檢查測試平台 (本週核心)

不要用眼睛看幾百行輸出。讓測試自己判分。

三個要素：

```
// 1. 參考模型 —— 獨立算一次標準答案
function [4:0] ref_model;
    input [3:0] x, z;  input [2:0] o;
    begin
        case (o)
            3'd0: ref_model = x + z;
            ...
        endcase
    end
endfunction

// 2. 自動比對
task check;
    begin
        #5;
        if (y === ref_model(a, b, op)) pass = pass + 1;
        else begin
            fail = fail + 1;
            $display("    [X] op=%0d a=%0d b=%0d → 得到 %0d，應該是 %0d", ...);
        end
    end
endtask

// 3. 最後報分
$display("    通過 %0d 組，失敗 %0d 組", pass, fail);
```

為什麼參考模型不能抄設計的寫法？抄了就是拿同一個錯誤比對同一個錯誤，永遠測不出來。參考模型要用不同的思路寫（或用行為級的簡單寫法）。

=== VS ==

	遇到 x / z
==	回傳 x（不確定）→ if 判斷不成立，bug 溜過去
===	嚴格比較，連 x 都要一模一樣

[!TIP] 測試平台一律用 === 和 !==。設計裡才用 ==。

四、本週四個範例

```
cd week05_simulation
```

📁 01_display_family — 四種印法 ★

```
iverilog -o sim.out 01_display_family.v 01_display_family_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 01_display_family.gtkw
```

親眼看到 `$display` 和 `$strobe` 在同一時間點印出不同的值。

📁 02_self_check — 自我檢查測試平台 ★★

```
iverilog -o sim.out 02_self_check.v 02_self_check_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 02_self_check.gtkw
```

一個 4 位元 ALU，窮舉 2048 組 + 亂數 500 組，全自動比對。

實測：

```
通過 2548 組，失敗 0 組
>>> 全部通過 ✓
```

這就是業界測試平台的樣子。之後你自己寫設計，測試都照這個模板。

📁 03_timescale — 時間單位

```
iverilog -o sim.out 03_timescale.v 03_timescale_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 03_timescale.gtkw
```

兩個不同頻率的時脈（100MHz / 25MHz）並排。**動手試**：把最上面的 ``timescale 1ns/1ps` 改成 `1us/1ns`，波形時間軸會慢 1000 倍。

📁 04_random_test — 亂數測試抓 bug ★

```
iverilog -o sim.out 04_random_test.v 04_random_test_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 04_random_test.gtkw
```

★ 這個 DUT 故意藏了一個 bug (`lt` 寫成 `a <= b` , 應該是 `a < b`) 。只有 `a == b` 時才會錯 —— 用眼睛幾乎看不出來。

實測：

```
[X] 第0次 a=10 b=10 | 得到 gt=0 eq=1 lt=1, 應該是 0 1 0
    通過 1706 組, 失敗 294 組
    >>> 抓到 bug! 第一次失敗在第 0 次
```

兩個關鍵技巧：

1. 固定亂數種子 `$random(seed)` → 失敗可以重現，不會這次錯下次對
2. 刻意加邊界值 (這裡強迫 `a == b`) → 大幅提高抓 bug 機率

五、練習題

練習 1

把 `04_random_test.v` 的 bug 修好 (`a <= b` → `a < b`) , 重跑確認全過。

練習 2 ★

幫第 4 週的 `ripple_adder` 寫一個完整的自我檢查測試：窮舉所有 8 位元 `a`、`b` 組合 (65536 組) + `cin`，看要跑多久。

練習 3

幫第 3 週的 `shift_reg` 寫自我檢查測試。提示：參考模型要自己維護一份「預期的 `q`」，每拍跟著移位。

練習 4 (思考)

為什麼下面這個測試「永遠不會失敗」，即使 DUT 是錯的？

```
if (y == golden) pass = pass + 1;  
else             fail = fail + 1;
```

(假設 `y` 有時候是 `x`)

► 看答案

六、本週檢核

- [] 四個範例都跑過
 - [] 說得出 `$display` 和 `$strobe` 差在哪，以及什麼時候會踩到
 - [] 能照著 02 的模板，自己寫一個自我檢查測試
 - [] 練習 1 修好 bug 並確認全過
 - [] 練習 4 答對
-

附註：關於 `sim` 這個指令

你可能在別的地方看過我寫的 `sim` 範例名稱。那是我自己做的批次檔（`sim.bat`），不是 Verilog 或 iverilog 的標準指令，教科書上查不到。

它做的事就是把上面那四行包起來、順便幫你檢查結束碼。用不用都可以，也可以刪掉。這份 README 裡給的全部都是原始指令。